

Machine Learning Applied to Quantitative Finance: Linear Regression, Logistic Classification, and Backtesting from First Principles

William Thompson

Department of Physics, Lancaster University

Abstract

A machine learning pipeline is developed from scratch using only NumPy and pandas, where no external ML frameworks are used, and applied progressively from a pedagogical advertising dataset to real financial market data. Linear regression with batch gradient descent is implemented and verified against known benchmark values, achieving a final benchmark prediction of $\hat{y} = 12.395$ for an input of $x = 16.5$ after 5,000 epochs at a learning rate of $\alpha = 0.001$. Extension to multi-feature regression ($D = 3$) yields $R^2 = 0.89$ on advertising sales data, with feature normalisation shown to be necessary for gradient descent convergence. Ridge regularisation identifies an optimal $\lambda = 0.04$ on a held-out validation set. The Capital Asset Pricing Model (CAPM) is fitted to daily log returns for AAPL, MSFT, and NVDA over 2018–2023, recovering statistically significant market betas ($\beta_{\text{NVDA}} = 1.77$, $p < 0.0001$) but no statistically significant alpha for any stock, consistent with the efficient market hypothesis. A logistic regression classifier trained on lagged technical features to predict next-day NVDA direction achieves $\sim 53\%$ accuracy on the test set, which is below the $\sim 55\%$ naive baseline of always predicting a down day, confirming that simple momentum and volatility features are insufficient to reliably exploit short-term price predictability. A long/cash backtest of the classifier produces an annualised Sharpe ratio of 0.90 on the test set versus 1.99 for buy-and-hold, but outperforms buy-and-hold on a risk-adjusted basis during the 2022 bear market (Sharpe -0.23 vs -0.34), suggesting the model functions as an unintentional defensive timing strategy.

Contents

1	Introduction	3
2	Theory	3
2.1	Linear Regression	3
2.1.1	Maximum Likelihood Justification	3
2.2	Multi-Feature Regression and Matrix Notation	4
2.3	Gradient Descent	4
2.3.1	Feature Normalisation	4
2.4	Ridge Regularisation	4
2.5	Capital Asset Pricing Model	5
2.6	Logistic Regression	5
2.7	Sharpe Ratio	5
3	Methods	6
3.1	Benchmark Implementation	6
3.2	Multi-Feature Regression	6
3.3	Financial Data	6
3.4	CAPM Regression	6
3.5	Direction Classifier	7
3.6	Backtest	7
4	Results	8
4.1	Benchmark Task: Linear Regression (D=1)	8
4.2	Multi-Feature Regression (D=3)	9
4.3	CAPM and Factor Analysis	10
4.4	Direction Classifier and Backtest	12
5	Conclusion	13

1 Introduction

Machine learning (ML) encompasses algorithms that learn patterns from data without being explicitly programmed to do so [1]. In supervised learning, the paradigm used throughout this project, a model is trained on labelled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $\mathbf{x}_i$ is a feature vector and y_i is a target label, and the goal is to learn a function $f : \mathbf{x} \mapsto y$ that generalises to unseen data [2].

The application of supervised learning to financial markets is an active area of quantitative research [7]. Regression methods are used to estimate asset risk exposures (the Capital Asset Pricing Model being a canonical example), while classification methods are used to predict discrete outcomes such as price direction. Both rest on the same mathematical foundation, this being the minimisation of a cost function via gradient descent, making them natural subjects for a unified implementation.

This report describes a project in four stages. Section 2 develops the mathematical background for all methods used. Section 3 describes the implementation. Section 4 presents and analyses the results at each stage, from benchmark verification through to backtesting on real market data. Section 5 draws conclusions and identifies limitations.

All code is implemented from scratch in Python using NumPy [8] and pandas [9]. No machine learning frameworks (scikit-learn, TensorFlow, PyTorch) are used at any point.

2 Theory

2.1 Linear Regression

Linear regression fits a linear model to data by minimising the mean squared error (MSE) between predictions and labels. For $D = 1$ (a single feature), the model is:

$$\hat{y} = wx + b \quad (1)$$

where w is the weight (slope) and b is the bias (intercept). The MSE cost function is:

$$\ell(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - wx_i - b)^2 \quad (2)$$

Minimising Eq. (2) by setting partial derivatives to zero yields the analytical solution [3]:

$$w^* = \frac{\text{Cov}(x, y)}{\text{Var}(x)}, \quad b^* = \bar{y} - w^* \bar{x} \quad (3)$$

This result, that the optimal regression slope is the ratio of the covariance of x and y to the variance of x , connects directly to the Capital Asset Pricing Model in Section 2.5.

2.1.1 Maximum Likelihood Justification

The MSE cost function is not an arbitrary choice. If the noise term $\epsilon_i = y_i - \hat{y}_i$ is assumed to be independently and identically distributed as $\mathcal{N}(0, \sigma^2)$, then the probability of observing the full dataset is:

$$p(\mathbf{y} \mid \mathbf{x}, w, b) = \prod_{i=1}^N \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(y_i - wx_i - b)^2}{2\sigma^2}\right) \quad (4)$$

Maximising the log-likelihood with respect to w and b is equivalent to minimising $\sum_i (y_i - wx_i - b)^2$, which is precisely the MSE cost. Least-squares regression is therefore maximum likelihood estimation under Gaussian noise.

2.2 Multi-Feature Regression and Matrix Notation

For $D > 1$ features, the model is expressed in matrix form. Define the feature matrix $\mathbf{X} \in \mathbb{R}^{N \times (D+1)}$ with a leading column of ones (absorbing the bias into the parameter vector), and the parameter vector $\boldsymbol{\theta} \in \mathbb{R}^{D+1}$. Then:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} \quad (5)$$

The gradient of the MSE cost with respect to $\boldsymbol{\theta}$ is:

$$\nabla_{\boldsymbol{\theta}} \ell = -\frac{2}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \quad (6)$$

Setting this to zero yields the *normal equation*:

$$\boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (7)$$

2.3 Gradient Descent

For large D , the matrix inversion in Eq. (7) is computationally expensive. Gradient descent provides an iterative alternative. Starting from $\boldsymbol{\theta} = \mathbf{0}$, parameters are updated at each epoch as:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} \ell \quad (8)$$

where $\alpha > 0$ is the learning rate. Repeated application of Eq. (8) converges to the global minimum of the MSE cost, which is a strictly convex quadratic bowl, provided α is sufficiently small.

2.3.1 Feature Normalisation

When features have different scales, the cost function becomes an elongated ellipsoid in parameter space: the gradient points primarily across the valley rather than toward the minimum. Gradient descent oscillates and requires a very small α to avoid divergence. Z-score normalisation resolves this by transforming each feature $x^{(j)}$ to have zero mean and unit standard deviation:

$$x^{(j)} \leftarrow \frac{x^{(j)} - \mu_j}{\sigma_j} \quad (9)$$

where μ_j and σ_j are computed from the training set only. Applying the same μ_j and σ_j to validation and test data prevents look-ahead bias.

2.4 Ridge Regularisation

With many features, the model may overfit the training data. Ridge regression (L2 regularisation) adds a penalty term to the cost function:

$$\ell_{\text{ridge}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^D w_j^2 \quad (10)$$

where $\lambda \geq 0$ controls regularisation strength and the bias term $b = \theta_0$ is excluded from the penalty. The analytical solution becomes:

$$\boldsymbol{\theta}_{\text{ridge}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda N \mathbf{I}_{\text{reg}})^{-1} \mathbf{X}^\top \mathbf{y} \quad (11)$$

where $\mathbf{I}_{\text{reg}}$ is the identity matrix with the top-left element set to zero. For $\lambda = 0$ this reduces to Eq. (7). As $\lambda \rightarrow \infty$, all weights are driven toward zero, eliminating model capacity. The optimal λ is chosen by minimising validation set cost.

2.5 Capital Asset Pricing Model

The CAPM models a stock's excess return as a linear function of the market's excess return [4]:

$$R_i = \alpha + \beta R_m + \epsilon_i \quad (12)$$

where R_i is the stock's daily log return, R_m is the market proxy (SPY), β measures the stock's sensitivity to market moves, and α is the excess return unexplained by market risk. From Eq. (3), the CAPM beta is:

$$\beta = \frac{\text{Cov}(R_i, R_m)}{\text{Var}(R_m)} \quad (13)$$

Statistical significance of α and β is assessed via t -statistics with $N - 2$ degrees of freedom. Finding statistically significant positive alpha (a return above what market risk justifies) is the central challenge of active investment management.

2.6 Logistic Regression

When the target label is binary ($y_i \in \{0, 1\}$), linear regression is inappropriate as it can produce predictions outside $[0, 1]$. Logistic regression applies the sigmoid function to the linear score:

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \quad z_i = \boldsymbol{\theta}^\top \mathbf{x}_i \quad (14)$$

where $\hat{p}_i \in (0, 1)$ is the predicted probability of $y_i = 1$. The binary cross-entropy cost function is:

$$\ell(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (15)$$

This penalises confident incorrect predictions more severely than MSE, and arises from maximum likelihood estimation under a Bernoulli noise model. The gradient of Eq. (15) has the same matrix structure as Eq. (6):

$$\nabla_{\boldsymbol{\theta}} \ell = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}) \quad (16)$$

so the same gradient descent update, Eq. (8), applies with $\hat{\mathbf{p}}$ in place of $\hat{\mathbf{y}}$.

2.7 Sharpe Ratio

The Sharpe ratio measures risk-adjusted return [5]:

$$S = \frac{\mathbb{E}[r - r_f]}{\sigma(r - r_f)} \quad (17)$$

where r is the strategy’s daily return and r_f is the risk-free rate. Setting $r_f = 0$ (negligible for daily returns) and annualising using 252 trading days per year:

$$S_{\text{annual}} = \frac{\bar{r}}{\sigma_r} \sqrt{252} \quad (18)$$

A long/cash strategy earns the asset return on days the model predicts an up day, and zero on days it predicts a down day:

$$r_t^{\text{strategy}} = \text{pos}_t \times r_t, \quad \text{pos}_t = \mathbf{1}[\hat{p}_t \geq 0.5] \quad (19)$$

3 Methods

3.1 Benchmark Implementation

The benchmark task uses the Advertising dataset [3], which contains 200 observations of TV, radio, and newspaper advertising spend (in \$k) and resulting product sales (in 1,000 units). For the D=1 benchmark, only radio spend is used as the input feature.

Gradient descent is initialised at $w = 0$, $b = 0$ and run for 5,000 epochs with $\alpha = 0.001$. Results are stored after each update so that the state after k gradient updates is available at index k . This indexing convention means the benchmark document’s “epoch N ” corresponds to array index $N + 1$ in the implementation.

3.2 Multi-Feature Regression

All three advertising features are used ($D = 3$). The feature matrix $\mathbf{X} \in \mathbb{R}^{200 \times 4}$ includes a leading column of ones. Data is randomly split into training (70%), validation (15%), and test (15%) sets using a fixed random seed for reproducibility, giving 140 training, 30 validation, and 30 test observations. Z-score normalisation is applied using training set statistics only (Eq. 9). The normal equation (Eq. 7) provides the analytical optimum; ridge regularisation (Eq. 11) is applied across $\lambda \in [0.01, 0.1]$ with the optimal value chosen by minimising validation cost.

3.3 Financial Data

Daily closing prices for AAPL, MSFT, NVDA, and the S&P 500 ETF (SPY) are downloaded from Yahoo Finance via the `yfinance` library for the period 2018-01-01 to 2023-12-31. Log returns are computed as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) \quad (20)$$

Log returns are preferred over simple returns because they are time-additive and more closely satisfy the stationarity assumption required by linear regression. Series are aligned to a common date index before analysis, yielding 1,508 common trading days.

3.4 CAPM Regression

For each stock, the CAPM regression (Eq. 12) is fitted using ordinary least squares via `scipy.stats.linregress`. Standard errors for α and β are computed analytically from the regression residuals, and t -statistics are evaluated against the t -distribution with $N - 2$ degrees of freedom for hypothesis testing.

3.5 Direction Classifier

Six lagged features are constructed for NVDA, each shifted by at least one day to prevent look-ahead bias:

- **market_return**: SPY log return, lagged 1 day
- **momentum_5**: NVDA 5-day cumulative return, lagged 1 day
- **momentum_21**: NVDA 21-day cumulative return, lagged 1 day
- **volatility_21**: NVDA 21-day rolling standard deviation, lagged 1 day
- **return_lag1**: NVDA return from 1 day ago
- **return_lag2**: NVDA return from 2 days ago

After the rolling-window features are constructed and rows containing resulting NaNs are dropped, 1,487 usable observations remain. Days with returns exceeding a threshold of $r_t > 0.5\%$ are labelled as up days ($y_t = 1$), otherwise down ($y_t = 0$). This threshold filters out noise from negligibly small moves. Data is split *chronologically* (no shuffling): training on 2018–2021, validation on 2021–2022, and testing on 2022–2023. Shuffling is explicitly avoided — using future data to train on past labels would constitute look-ahead bias. The classifier is trained for 100,000 epochs with $\alpha = 0.01$.

3.6 Backtest

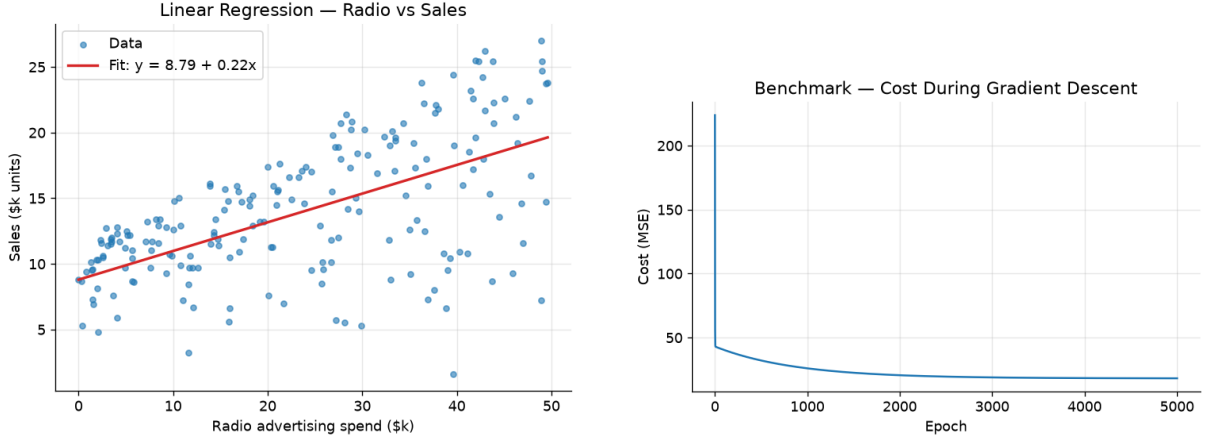
A long/cash strategy is simulated on the validation and test sets using Eq. (19). The benchmark is a buy-and-hold strategy on NVDA. Cumulative log returns are computed as:

$$C_t = \sum_{s=1}^t r_s^{\text{strategy}} \quad (21)$$

and the total return over the period is $e^{C_T} - 1$. All backtests exclude transaction costs and assume perfect execution.

4 Results

4.1 Benchmark Task: Linear Regression (D=1)



(a) Fitted regression line overlaid on the radio spend vs sales scatter plot. The line of best fit is $\hat{y} = 8.79 + 0.22x$, consistent with a positive but moderate relationship between radio spend and sales.

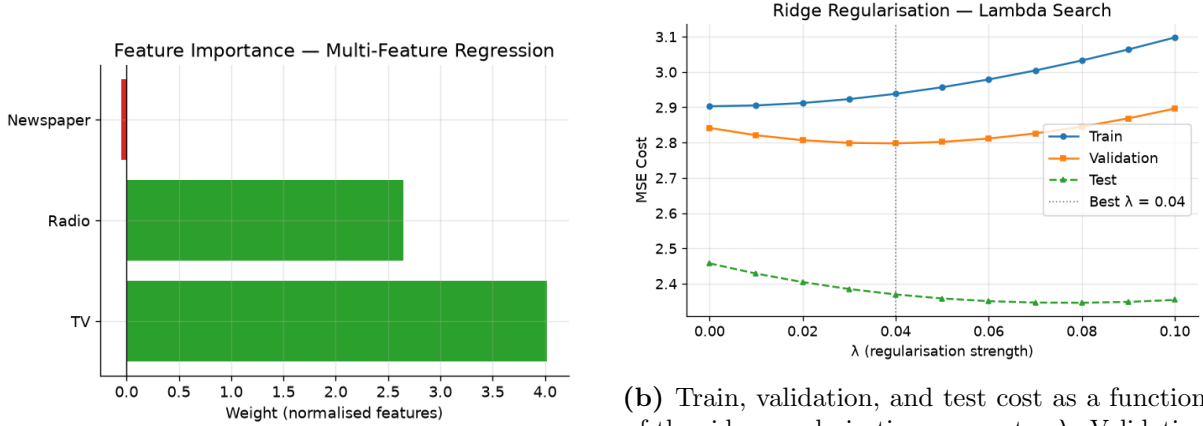
(b) MSE cost function as a function of epoch number during gradient descent with $\alpha = 0.001$. The cost decreases monotonically, dropping steeply in the first ~ 100 epochs before flattening as the minimum is approached.

Figure 1: Benchmark task results: simple linear regression of radio advertising spend on product sales ($N = 200$, $\alpha = 0.001$).

The gradient descent implementation is verified against three known benchmark values at $\alpha = 0.001$. After one gradient update (document epoch 0), the cost is $\ell = 92.32$, consistent with the expected value of 92.3208. After 800 updates (epoch 800), $\ell = 27.99$ versus the expected 27.9919. After 5,000 epochs, an input of $x = 16.5$ yields a predicted sales value of $\hat{y} = 12.395$. All three values agree with the benchmark specification to four significant figures, confirming the correctness of the gradient descent implementation.

The fitted regression line is shown in Fig. 1a, with final parameters $w = 0.218$ and $b = 8.791$. The cost convergence curve (Fig. 1b) exhibits the classic gradient descent profile: rapid initial descent followed by asymptotic approach to the minimum. The cost at epoch 0 ($\ell = 223.7$) is consistent with $\frac{1}{N} \sum y_i^2$, which is the expected MSE when $w = 0$ and $b = 0$ so that all predictions are zero.

4.2 Multi-Feature Regression ($D=3$)



(a) Normalised feature weights from the analytical solution at $\lambda = 0$. TV spend is the dominant driver of sales, radio is secondary, and newspaper spend has a weight of approximately zero.

(b) Train, validation, and test cost as a function of the ridge regularisation parameter λ . Validation cost is minimised at $\lambda = 0.04$, with test cost following a similar but not identical trend; motivating the use of a separate validation set for hyperparameter selection.

Figure 2: Multi-feature regression on the full advertising dataset ($D = 3$, $N = 200$, train/val/test split 70/15/15).

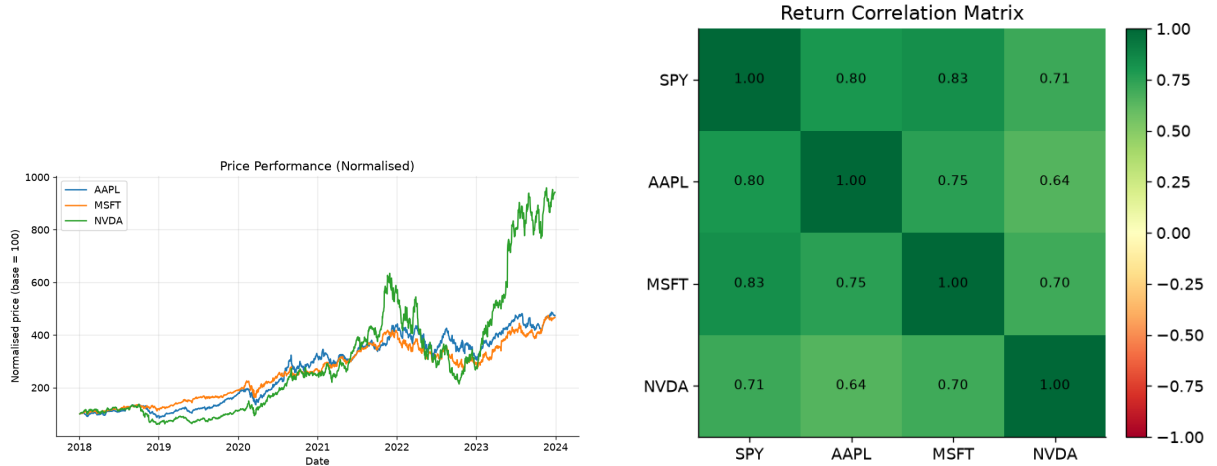
The analytical solution (normal equation) achieves a training MSE of 2.90 and a validation MSE of 2.84, corresponding to $R^2 = 0.89$ on the training set. Gradient descent converges to the same values after 200,000 epochs, confirming implementation correctness.

The normalised feature weights (Fig. 2a) reveal that TV advertising spend is the strongest driver of sales, with a weight of 4.02 per normalised standard deviation. Radio is second at 2.65. Newspaper spend has a weight of -0.05 , statistically indistinguishable from zero. The finding that newspaper advertising contributes negligible marginal sales above what TV and radio already explain, would not be recoverable from $D = 1$ regression, where all three features contribute to the radio coefficient through their mutual correlations.

Feature normalisation is found to be essential. Without it, gradient descent diverges within a handful of epochs at $\alpha = 0.001$ due to TV values (range 0–300) producing gradients several times larger than radio values (range 0–50), causing the loss landscape to be a highly elongated ellipsoid. Z-score normalisation transforms this into an approximately spherical bowl, enabling stable convergence.

The ridge regularisation search (Fig. 2b) shows validation cost minimised at $\lambda = 0.04$, improving validation MSE from 2.84 (unregularised) to 2.80 and test MSE from 2.46 to 2.37, a 3.6% improvement on the held-out test set. The performance improvement from regularisation, despite the model not exhibiting obvious overfitting (train $\approx$ val cost at $\lambda = 0$), reflects the stabilisation of coefficient estimates in the presence of mild multicollinearity between advertising spend variables.

4.3 CAPM and Factor Analysis



(a) Normalised closing prices (base = 100 at 2018-01-01) for AAPL, MSFT, and NVDA over 2018–2023. NVDA exhibits the most extreme price trajectory, driven by the AI semiconductor narrative from 2022 onwards.

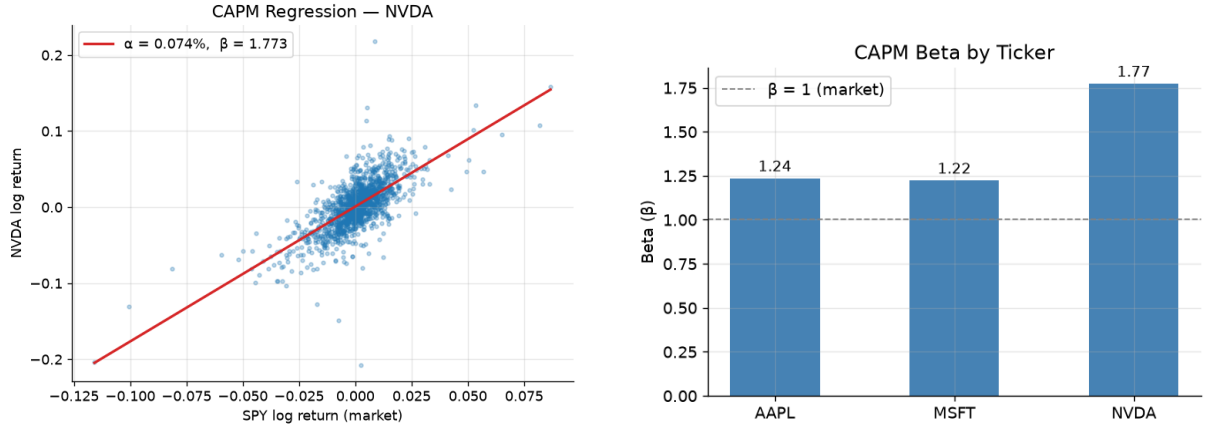
(b) Pearson correlation matrix of daily log returns for all four series. All pairwise correlations are positive, indicating that diversification within this equity universe cannot eliminate systematic market risk.

Figure 3: Financial data summary for AAPL, MSFT, NVDA, and SPY over 1,508 common trading days (2018-01-01 to 2023-12-31).

Table 1 summarises the CAPM regression results for the three stocks. All beta estimates are statistically significant at the $p < 0.0001$ level, with t -statistics exceeding 38 in all cases. No alpha is statistically significant at the conventional $p < 0.05$ threshold, consistent with the efficient market hypothesis for large-cap, liquid US equities over a five-year horizon.

Table 1: CAPM regression results for AAPL, MSFT, and NVDA against SPY as the market proxy, using 1,508 daily log returns over 2018–2023. Alpha is reported as a daily percentage. p -values are two-sided against the null hypothesis that the coefficient is zero.

Ticker	α (%)	β	t_α	t_β	p_α	p_β	R^2
AAPL	0.048	1.236	1.550	51.38	0.121	< 0.0001	0.637
MSFT	0.048	1.223	1.764	57.85	0.078	< 0.0001	0.690
NVDA	0.074	1.773	1.257	38.77	0.209	< 0.0001	0.500



(a) CAPM scatter plot for NVDA. The regression line $R_{\text{NVDA}} = 0.074\% + 1.773R_{\text{SPY}}$ is shown in red. The wide scatter about the regression line ($R^2 = 0.50$) indicates substantial idiosyncratic variance not explained by market movements.

(b) CAPM beta values for the three stocks. NVDA has the highest beta ($\beta = 1.77$), amplifying daily market moves by 77%. AAPL and MSFT have similar betas of approximately 1.23, indicating near-proportional market sensitivity.

Figure 4: CAPM regression results. Market proxy is SPY (S&P 500 ETF).

NVDA has the highest beta ($\beta = 1.77$), consistent with its annualised volatility of 51.3% versus the SPY baseline of 20.5%. However, NVDA's $R^2 = 0.50$ is substantially lower than AAPL's 0.64 or MSFT's 0.69, despite its larger beta. This apparent contradiction is resolved by noting that R^2 measures the *fraction* of variance explained by the market, while β measures the *magnitude* of market sensitivity. NVDA has both high market sensitivity and high idiosyncratic variance — the latter driven by company-specific factors such as the AI/semiconductor narrative — so the market explains only half its daily moves even though those moves are amplified.

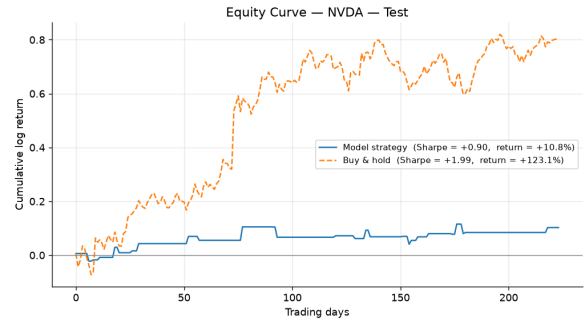
The correlation matrix (Fig. 3b) confirms that all stocks are positively correlated with SPY (correlations 0.71–0.83) and with each other. All correlations are positive, meaning the portfolio retains systematic (market) risk regardless of weighting. The lowest pairwise correlation is NVDA–AAPL at 0.64, reflecting NVDA's more idiosyncratic behaviour.

Return distributions for all stocks exhibit negative skewness and excess kurtosis (kurtosis > 0 relative to the Gaussian baseline), consistent with the well-documented phenomenon of fat tails in equity returns [6]. SPY shows the largest kurtosis (11.9), as the aggregate index is subject to sudden market-wide shocks.

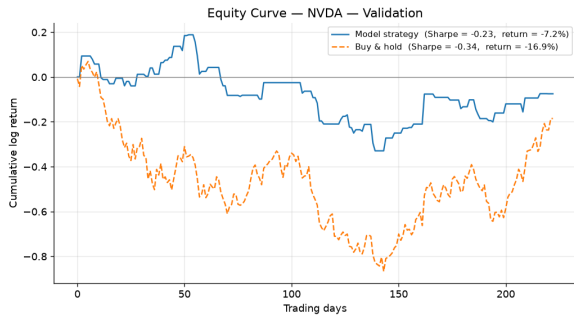
4.4 Direction Classifier and Backtest



(a) Cross-entropy loss during logistic regression training ($\alpha = 0.01$, 100,000 epochs). The loss converges to ~ 0.682 , close to $-\ln(0.5) = 0.693$, indicating the model outputs near-uniform probabilities regardless of input features.



(b) Equity curves for the model strategy and buy-and-hold NVDA on the test set (2023). Buy-and-hold substantially outperforms the model strategy (Sharpe 1.99 vs 0.90), as the model sits in cash through much of NVDA’s AI-driven bull run.



(c) Equity curves on the validation set (2021–2022), which covers the 2022 bear market. The model strategy outperforms buy-and-hold on a risk-adjusted basis (Sharpe -0.23 vs -0.34) by avoiding much of the 2022 drawdown through cash positions.

	Strategy	B&H
<i>Validation (2021–22)</i>		
Sharpe	-0.23	-0.34
Total return	-7.2%	-16.9%
<i>Test (2023)</i>		
Sharpe	$+0.90$	$+1.99$
Total return	$+10.8\%$	$+123.1\%$

(d) Summary of backtest performance. The model reduces drawdown in bear markets (validation) but misses bull market upside (test), consistent with a defensive timing profile rather than a return-enhancing strategy.

Figure 5: Logistic regression direction classifier and backtest results for NVDA (2018–2023). Training set: 2018–2021; validation set: 2021–2022; test set: 2023.

The class balance of the training set is 45.2% up days (using the 0.5% threshold), so the naive baseline of always predicting down achieves $\sim 54.8\%$ accuracy. The trained classifier achieves 56.5% accuracy on the training set, 49.3% on validation, and 52.7% on the test set. On the test set the classifier therefore falls slightly *below* the naive down-always baseline, and its apparent edge on the training set does not generalise. The convergence of the training loss to $\sim 0.682 \approx \ln 2$ (Fig. 5a) confirms that the model has not learned a meaningful decision boundary from the features — the output is approximately $\hat{p} \approx 0.45$ for all inputs, reflecting the class prior. Consistent with this, test recall is only 13.2% at a precision of 50.0%: the model rarely commits to an up prediction, and when it does it is right only about half the time.

This null result is consistent with the efficient market hypothesis for large-cap liquid equities: if simple lagged price features reliably predicted next-day direction, rational investors would exploit this signal until it disappeared. The test classification accuracy of 52.7% is not statistically distinguishable from chance given the sample size.

The backtest results (Figs. 5b and 5c, Table 5d) reveal a nuanced performance profile. On the test set (2023, NVDA’s AI-driven bull run), the model strategy underperforms buy-and-hold severely: Sharpe 0.90 versus 1.99, total return 10.8% versus 123.1%. The model remains in cash through much of the rally (invested only 12% of days), missing the upside.

On the validation set (2021–2022, including the 2022 bear market), the model outperforms buy-and-hold on a risk-adjusted basis: Sharpe -0.23 versus -0.34 , with the strategy’s maximum drawdown substantially smaller than buy-and-hold’s (total return -7.2% versus -16.9%). This improvement arises not from genuine predictive skill but from *reduced exposure*: by predicting down on many days and holding cash (invested only 33% of days), the strategy avoids participating in the worst of the 2022 decline. The model functions as an unintentional defensive timing strategy — reducing volatility at the cost of missing upside.

5 Conclusion

A complete machine learning pipeline has been implemented from scratch and applied to both a pedagogical advertising dataset and real financial market data. The core result of each stage is summarised below.

Linear regression with gradient descent was verified against known benchmark values to four significant figures, confirming the correctness of the implementation. The extension to $D = 3$ features demonstrated that feature normalisation is *necessary* for gradient descent convergence when features differ in scale, and that ridge regularisation reduces out-of-sample MSE by 3.6% even in the absence of obvious overfitting. The newspaper advertising feature was found to carry negligible predictive information once TV and radio spend are controlled for — a finding recoverable only from multi-feature regression.

CAPM regression on five years of daily returns recovered statistically significant market betas for all three stocks ($p < 0.0001$) but found no statistically significant alpha at the 5% level. This is consistent with market efficiency for large-cap US equities and highlights that any apparent outperformance must be evaluated rigorously against the multiple testing problem: with sufficiently many stocks and time periods tested, spurious significance is inevitable.

The direction classifier produced near-chance accuracy on the test set, in fact marginally below the naive down-always baseline, consistent with the difficulty of predicting short-term equity price movements from simple lagged features. The backtest demonstrated that the classifier, despite its weak predictive performance, functions as a risk-reduction tool during bear markets through its tendency to hold cash — achieving a better Sharpe ratio than buy-and-hold during 2022 at the cost of missing the 2023 bull run.

The primary limitation of this work is the restricted feature set for the classifier. Professional quant models incorporate options-implied volatility, order book data, earnings surprise, analyst revisions, and macroeconomic factors — none of which are captured by lagged price features alone. A natural extension would be to incorporate cross-sectional factor models (e.g. Fama-French) as features, or to replace the logistic classifier with a gradient-boosted decision tree, which can capture non-linear interactions between features that logistic regression cannot. A further limitation is the absence of transaction costs in the

backtest: for a strategy that trades frequently, bid-ask spreads and market impact would substantially reduce net returns.

References

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [2] T. Hastie, R. Tibshirani, and J. H. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, New York, 2009. Available: <https://www.statlearning.com/>
- [3] A. Burkov, *The Hundred-Page Machine Learning Book*. Andriy Burkov, 2019.
- [4] W. F. Sharpe, “Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk,” *Journal of Finance*, vol. 19, no. 3, pp. 425–442, 1964.
- [5] W. F. Sharpe, “Mutual Fund Performance,” *Journal of Business*, vol. 39, no. 1, pp. 119–138, 1966.
- [6] E. F. Fama, “The Behavior of Stock-Market Prices,” *Journal of Business*, vol. 38, no. 1, pp. 34–105, 1965.
- [7] M. López de Prado, *Advances in Financial Machine Learning*. Wiley, New York, 2018.
- [8] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [9] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.